

Tutorial Passo a Passo: Construindo uma API RESTful de Gestão de Vagas com Spring Boot 4 do Zero

Elaborei este tutorial foi especialmente para meus alunos que **nunca utilizaram o Spring Boot** ou está dando os primeiros passos no desenvolvimento backend com Java.

Aqui você aprenderá como construir, do absoluto zero, uma API RESTful completa de **Gestão de Vagas de Emprego** utilizando as melhores práticas do mercado.

Sumário

1. Visão Geral e Conceitos Básicos
2. Pré-requisitos e Ferramentas
3. Passo 1: Criando o Projeto e Configurando o Maven (`pom.xml`)
4. Passo 2: Criando a Estrutura de Pacotes
5. Passo 3: Configurando a Conexão com o Banco de Dados
6. Passo 4: Criando a Entidade de Domínio (`Vaga.java`)
7. Passo 5: Criando o Repositório (`VagaRepository.java`)
8. Passo 6: Criando os DTOs e Mapeamento com MapStruct
9. Passo 7: Criando a Camada de Tratamento Global de Erros
10. Passo 8: Criando a Camada de Serviço (`VagaService.java`)
11. Passo 9: Criando o Controller REST (`VagaController.java`)
12. Passo 10: Documentando a API com Swagger UI
13. Passo 11: Executando e Testando os Endpoints
14. Conclusão e Próximos Passos

1. Visão Geral e Conceitos Básicos

O que é o Spring Boot?

O **Spring Boot** é um framework Java que facilita imensamente a criação de aplicações prontas para produção. Ele abstrai configurações complexas, fornece um servidor web embutido (como o Tomcat) e permite que você rode sua aplicação como um simples arquivo `.jar` ou através do terminal.

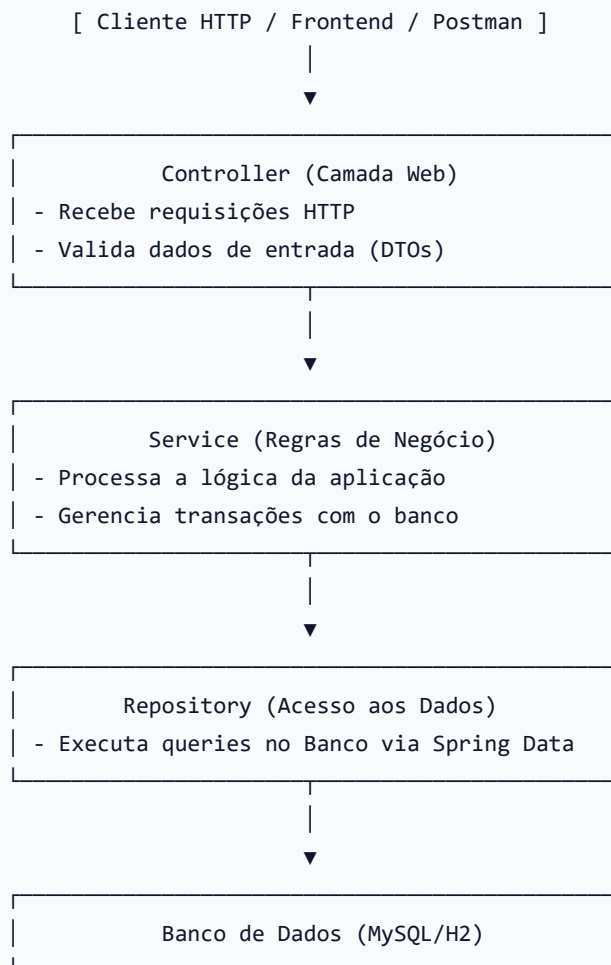
O que é uma API RESTful?

Uma API RESTful permite que diferentes sistemas se comuniquem através do protocolo HTTP. Os dados são trocados no formato **JSON**, utilizando verbos padrão:

- `GET` : Buscar/listar dados.
- `POST` : Criar novos dados.
- `PUT` : Atualizar dados existentes.
- `DELETE` : Remover dados.

Arquitetura em Camadas (Layered Architecture)

Para manter o código organizado, dividimos a aplicação em camadas bem definidas:



2. Pré-requisitos e Ferramentas

Antes de iniciar, certifique-se de ter instalado em sua máquina:

1. **JDK 21** (Java Development Kit versão 21 ou superior).
 2. **Maven 4.X+** (gerenciador de dependências Java).
 3. **IDE de sua preferência:** VS Code (com extensões Java/Spring) ou IntelliJ IDEA.
 4. **Servidor MySQL 8.0+** (ou você pode usar o banco H2 em memória durante os testes).
 5. **Cliente HTTP para testes:** Swagger UI (já incluído no projeto), Postman ou Insomnia.
-

Passo 1: Criando o Projeto e Configurando o Maven (`pom.xml`)

Você pode gerar a estrutura inicial no site [Spring Initializr](#):

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** 4.1.1
- **Group:** `com.lab.jp`
- **Artifact:** `gestao-vagas`
- **Packaging:** Jar
- **Java:** 17

Dependências necessárias

Abra o arquivo [`pom.xml`] na raiz do projeto. Ele gerencia as bibliotecas que o projeto utiliza. Garanta que o arquivo contenha as dependências a seguir:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/> <!-- Lookup parent from repository -->
    </parent>
    <groupId>com.lab.jpas</groupId>
    <artifactId>gestao-vagas</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>gestao-vagas</name>
    <description>gestao-vagas</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>

```

```

        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct</artifactId>
        <version>1.6.3</version>
    </dependency>
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>3.1.0</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>

```

```

        <annotationProcessorPaths>
            <path>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
            </path>
            <path>
                <groupId>org.mapstruct</groupId>
                <artifactId>mapstruct-processor</artifactId>
                <version>1.6.3</version>
            </path>
        </annotationProcessorPaths>
    </configuration>
</execution>
<execution>
    <id>default-testCompile</id>
    <phase>test-compile</phase>
    <goals>
        <goal>testCompile</goal>
    </goals>
    <configuration>
        <annotationProcessorPaths>
            <path>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
            </path>
            <path>
                <groupId>org.mapstruct</groupId>
                <artifactId>mapstruct-processor</artifactId>
                <version>1.6.3</version>
            </path>
        </annotationProcessorPaths>
    </configuration>
</execution>
</executions>
</plugin>
</plugins>
</build>

</project>

```

Passo 2: Criando a Estrutura de Pacotes

No Java, os arquivos fonte devem ficar organizados dentro do diretório

```
src/main/java/com/lab/jpa/gestaovagas/
```

Crie os seguintes pacotes (pastas) dentro da sua IDE:

```
src/main/java/com/lab/jpa/gestaovagas/
├─ config/          <-- Configurações (ex: Swagger)
├─ controller/      <-- Controllers REST (Endpoints HTTP)
├─ domain/
│   └─ model/       <-- Entidades JPA (Tabelas do Banco)
├─ dto/             <-- Data Transfer Objects (Records de Entrada/Saída)
├─ exception/       <-- Manipulação de Erros Globais
├─ mapper/          <-- Interfaces de Conversão do MapStruct
├─ repository/      <-- Interfaces de acesso ao Banco de Dados
├─ service/         <-- Camada com a Lógica de Negócio
└─ GestaoVagasApplication.java <-- Classe Principal
```

A Classe Principal da Aplicação

No arquivo [`GestaoVagasApplication.java`] na raiz de `com.lab.jpa.gestaovagas` :

```
package com.lab.jpa.gestaovagas;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GestaoVagasApplication {

    public static void main(String[] args) {
        // Inicializa o servidor web embutido (Tomcat na porta 8080)
        SpringApplication.run(GestaoVagasApplication.class, args);
    }
}
```

[NOTA] A anotação `@SpringBootApplication` avisa ao Spring para procurar componentes, realizar configurações automáticas e inicializar o projeto.

Passo 3: Configurando a Conexão com o Banco de Dados

1. Configurando o arquivo `application.properties`

Abra o arquivo [`src/main/resources/application.properties`] e defina as propriedades de conexão:

```
# Configuração de Conexão com o Banco de Dados MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/db_vagas?
createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Configuração do Hibernate (JPA)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Caminhos do Swagger UI / OpenAPI
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.api-docs.path=/v3/api-docs

# Configurações Padrão de Paginação
spring.data.web.pageable.default-page-size=10
spring.data.web.pageable.max-page-size=100
```

[DICA:] `spring.jpa.hibernate.ddl-auto=update` faz com que o Hibernate crie ou atualize as tabelas do banco de dados automaticamente de acordo com as classes Java!

Passo 4: Criando a Entidade de Domínio (`Vaga.java`)

A entidade representa a tabela do banco de dados em formato de classe Java. Crie o arquivo [`Vaga.java`] no pacote `com.lab.jpj.gestaovagas.domain.model` :

```

package com.lab.jpa.gestaovagas.domain.model;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "BVagas")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public class Vaga {
    @Id
    @GeneratedValue (strategy = GenerationType.UUID)
    @EqualsAndHashCode.Include
    private UUID id;

    @Column(nullable = false, length = 100)
    private String titulo;

    @Column(columnDefinition = "TEXT")
    private String descricao;

    @Column(nullable = false)
    private double salario;

    @Column(name = "data_criacao", updatable = false)
    private LocalDateTime dataCriacao;

    @PrePersist
    public void onCreate(){
        if(this.dataCriacao == null){
            this.dataCriacao = LocalDateTime.now();
        }
    }
}

```

Explicação das Anotações:

- `@Entity` : Marca a classe como uma Entidade JPA (tabela no banco).
- `@Table(name = "BVagas")` : Define o nome da tabela no banco de dados.

- `@Id` e `@GeneratedValue(strategy = GenerationType.UUID)` : Define a chave primária usando o padrão seguro UUID (128 bits).
- `@Column` : Personaliza detalhes da coluna no banco (tamanho, obrigatoriedade, etc.).
- `@PrePersist` : Método executado automaticamente antes de salvar no banco para preencher a data de criação.
- Anotações do **Lombok** (`@Getter` , `@Setter` , `@Builder` , etc.): Geram getters, setters e construtores automaticamente em tempo de compilação.

Passo 5: Criando o Repositório (`VagaRepository.java`)

O Repositório é responsável por comunicar a aplicação com o banco de dados. O Spring Data JPA fornece métodos prontos como `save()`, `findById()`, `findAll()` e `deleteById()`.

Crie o arquivo (interface) [`VagaRepository.java`] no pacote `com.lab.jpa.gestaovagas.repository`:

```
package com.lab.jpa.gestaovagas.repository;

import com.lab.jpa.gestaovagas.domain.model.Vaga;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.UUID;

public interface VagaRepository extends JpaRepository<Vaga, UUID> {

}
```

Passo 6: Criando os DTOs e Mapeamento com MapStruct

Por boas práticas de arquitetura e segurança, **nunca devemos expor nossas Entidades JPA diretamente nos endpoints HTTP**. Em vez disso, utilizamos **DTOs (Data Transfer Objects)**.

1. DTO de Requisição (`VagaRequestDTO.java`)

Crie o arquivo [`VagaRequestDTO.java`] no pacote `com.lab.jpa.gestaovagas.dto` :

```
package com.lab.jpa.gestaovagas.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

public record VagaRequestDTO(
    @NotBlank(message = "O título é obrigatório.")
    @Size(max = 100, message = "O título deve ter no máximo 100 caracteres.")
    String titulo,
    String descricao,

    @NotNull(message = "O salário é obrigatório.")
    @Positive(message = "O salário deve ser um valor positivo.")
    Double salario
) {
}
```

[NOTA] Utilizamos **Java Records**, um recurso do Java 14+ ideal para objetos imutáveis de transferência de dados. As anotações `@NotBlank`, `@Size`, `@NotNull` e `@Positive` realizam validação automática.

2. DTO de Resposta (`VagaResponseDTO.java`)

Crie o arquivo [`VagaResponseDTO.java`] no pacote `com.lab.jpa.gestaovagas.dto` :

```

package com.lab.jpa.gestaovagas.dto;
import java.time.LocalDateTime;
import java.util.UUID;

public record VagaResponseDTO(
    UUID id,
    String titulo,
    String descricao,
    Double salario,
    LocalDateTime dataCriacao
) {
}

```

3. Mapeador MapStruct (`VagaMapper.java`)

O MapStruct converte `VagaRequestDTO` -> `Vaga` e `Vaga` -> `VagaResponseDTO` de forma automática e extremamente rápida.

Crie o arquivo [`VagaMapper.java`] no pacote `com.lab.jpa.gestaovagas.mapper` :

```

package com.lab.jpa.gestaovagas.mapper;

import com.lab.jpa.gestaovagas.domain.model.Vaga;
import com.lab.jpa.gestaovagas.dto.VagaRequestDTO;
import com.lab.jpa.gestaovagas.dto.VagaResponseDTO;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

@Mapper(componentModel = "spring")
public interface VagaMapper {

    VagaResponseDTO toDTO(Vaga entity);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "dataCriacao", ignore = true)
    Vaga toEntity(VagaRequestDTO dto);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "dataCriacao", ignore = true)
    void updateEntityFromDTO(VagaRequestDTO dto, @MappingTarget Vaga entity);
}

```

Passo 7: Criando a Camada de Tratamento Global de Erros

Quando ocorre um erro (ex: ID não encontrado ou validação com erro), a API deve responder com uma mensagem JSON limpa e um código de status HTTP adequado (404, 400, 500).

1. Exceção Customizada (`ResourceNotFoundException.java`)

Crie o arquivo [`ResourceNotFoundException.java`] no pacote `com.lab.jpa.gestaovagas.exception` :

```
package com.lab.jpa.gestaovagas.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

2. Estrutura da Resposta de Erro (`ErrorResponse.java`)

Crie o arquivo [`ErrorResponse.java`] no pacote `com.lab.jpa.gestaovagas.exception` :

```
package com.lab.jpa.gestaovagas.exception;

import com.fasterxml.jackson.annotation.JsonFormat;
import java.time.LocalDateTime;

public record ErrorResponse(
    @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM dd'T'HH:mm:ss")
    LocalDateTime timestamp,
    int status,
    String error,
    Object message,
    String pat
) {
}
```

3. Handler Global (`GlobalExceptionHandler.java`)

Crie o arquivo [`GlobalExceptionHandler.java`] no pacote `com.lab.jpa.gestaovagas.exception` :

```

package com.lab.jp.gestaovagas.exception;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {
    //Trata erro de recursos nao encontrados (HTTP 404)
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex,
        HttpServletRequest request) {
        ErrorResponse errorResponse = new ErrorResponse(
            LocalDateTime.now(),
            HttpStatus.NOT_FOUND.value(),
            "Recurso não encontrado",
            ex.getMessage(),
            request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(errorResponse);
    }

    // Trata erros de validação de campos do DTO (HTTP 400)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
        handleValidationErrors(MethodArgumentNotValidException ex, HttpServletRequest request) {
        Map<String, String> errors = new HashMap<>();
        for (FieldError fieldError: ex.getBindingResult().getFieldErrors()) {
            errors.put(fieldError.getField(), fieldError.getDefaultMessage());
        }
        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            "Erro de validação",
            errors,
            request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }

    // Trata qualquer outra exceção genérica (HTTP 500)
    @ExceptionHandler(Exception.class)

```

```
public ResponseEntity<ErrorResponse> handleGeneralException(Exception ex,
HttpServletRequest request) {
    ErrorResponse error = new ErrorResponse(
        LocalDateTime.now(),
        HttpStatus.INTERNAL_SERVER_ERROR.value(),
        "Erro interno no servidor",
        ex.getMessage(),
        request.getRequestURI()
    );
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
}
```

Passo 8: Criando a Camada de Serviço (`VagaService.java`)

A camada de serviço contém todas as regras de negócio da aplicação. Ela coordena as chamadas ao repositório e ao mapeador.

Crie o arquivo [`VagaService.java`] no pacote `com.lab.jp.gestaovagas.service` :

```

package com.lab.jpa.gestaovagas.service;

import com.lab.jpa.gestaovagas.domain.model.Vaga;
import com.lab.jpa.gestaovagas.dto.VagaRequestDTO;
import com.lab.jpa.gestaovagas.dto.VagaResponseDTO;
import com.lab.jpa.gestaovagas.exception.ResourceNotFoundException;
import com.lab.jpa.gestaovagas.mapper.VagaMapper;
import com.lab.jpa.gestaovagas.repository.VagaRepository;

import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.UUID;

@Service
@RequiredArgsConstructor
public class VagaService {
    private final VagaRepository repository;
    private final VagaMapper mapper;

    @Transactional(readOnly = true)
    public Page<VagaResponseDTO> listarTodas(Pageable pageable) {
        return repository.findAll(pageable).map(mapper::toDTO);
    }

    @Transactional
    public VagaResponseDTO salvar(VagaRequestDTO dto) {
        Vaga vaga = mapper.toEntity(dto);
        Vaga saved = repository.save(vaga);
        return mapper.toDTO(saved);
    }

    @Transactional(readOnly = true)
    public VagaResponseDTO buscarPorId(UUID id) {
        Vaga vaga = repository.findById(id)
            .orElseThrow(() ->
                new ResourceNotFoundException("Vaga não encontrada com o ID: " +
id));
        return mapper.toDTO(vaga);
    }

    @Transactional
    public VagaResponseDTO atualizar(UUID id, VagaRequestDTO dto) {
        Vaga vaga = repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Vaga não encontrada com o
ID: " + id));

```

```
        mapper.updateEntityFromDTO(dto, vaga);
        Vaga updated = repository.save(vaga);
        return mapper.toDTO(updated);
    }

    @Transactional
    public void remover(UUID id) {
        if (!repository.existsById(id)) {
            throw new ResourceNotFoundException("Vaga não encontrada com o ID: " + id);
        }
        repository.deleteById(id);
    }
}
```

Passo 9: Criando o Controller REST (`VagaController.java`)

O Controller é a porta de entrada da nossa API. Ele mapeia os caminhos HTTP (`/api/vagas`) para métodos Java.

Crie o arquivo [`VagaController.java`] no pacote `com.lab.jp.gestaovagas.controller` :

```

package com.lab.jpa.gestaovagas.controller;

import com.lab.jpa.gestaovagas.dto.VagaRequestDTO;
import com.lab.jpa.gestaovagas.dto.VagaResponseDTO;
import com.lab.jpa.gestaovagas.service.VagaService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.UUID;

@RestController
@RequestMapping("/api/vagas")
@RequiredArgsConstructor
@Tag(name = "Vagas", description = "API RESTful para Gestao de Vagas de Emprego")
public class VagaController {
    private final VagaService service;

    @PostMapping
    @Operation(summary = "Criar nova vaga", description = "Cadastra uma oportunidade de vaga de emprego")
    public ResponseEntity<VagaResponseDTO> criar(@Valid @RequestBody VagaRequestDTO dto) {
        VagaResponseDTO vagaSalva = service.salvar(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(vagaSalva);
    }

    @GetMapping
    @Operation(summary = "Listar todas as vagas com paginação", description = "Retorna uma página de vagas registradas com ordenação padrão por título asc.")
    public ResponseEntity<Page<VagaResponseDTO>> listar(@PageableDefault(page = 0, size = 10, sort = "titulo", direction = Sort.Direction.ASC) Pageable pageable) {
        Page<VagaResponseDTO> vagas = service.listarTodas(pageable);
        return ResponseEntity.ok(vagas);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Obter vaga por ID", description = "Recupera os detalhes de uma vaga específica informando o UUID.")
    public ResponseEntity<VagaResponseDTO> obter(@PathVariable UUID id) {
        VagaResponseDTO vaga = service.buscarPorId(id);
        return ResponseEntity.ok(vaga);
    }
}

```

```
@PutMapping("/{id}")
@Operation(summary = "Atualizar vaga", description = "Atualiza completamente as
informações de uma vaga existente.")
public ResponseEntity<VagaResponseDTO> atualizar(@PathVariable UUID id, @Valid
@RequestBody VagaRequestDTO dto) {
    VagaResponseDTO vagaAtualizada = service.atualizar(id, dto);
    return ResponseEntity.ok(vagaAtualizada);
}

@DeleteMapping("/{id}")
@Operation(summary = "Deletar vaga", description = "Remove o registro de uma vaga do
sistema.")
public ResponseEntity<Void> deletar(@PathVariable UUID id) {
    service.remover(id);
    return ResponseEntity.noContent().build();
}
}
```

Passo 10: Documentando a API com Swagger UI

Para configurar os títulos e informações no Swagger UI, crie o arquivo [`SwaggerConfig.java`] no pacote `com.lab.jp.gestaovagas.config`:

```
package com.lab.jp.gestaovagas.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.info.License;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class SwaggerConfig {
    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("API de Gestão de Vagas")
                .description("API RESTful corporativa desenvolvida para gerenciamento de vagas de emprego.")
                .version("1.0.0")
                .contact(new Contact()
                    .name("Suporte Gestão de Vagas")
                    .email("suporte@gestaovagas.com"))
                .license(new License()
                    .name("Apache 2.0")
                    .url("https://www.apache.org/licenses/LICENSE 2.0")));
    }
}
```

Passo 11: Executando e Testando os Endpoints

1. Iniciando a Aplicação

Abra o terminal na pasta raiz do projeto e execute:

- **Linux / macOS:**

```
./mvnw spring-boot:run
```

- **Windows (CMD ou PowerShell):**

```
mvnw.cmd spring-boot:run
```

Após a compilação, o Spring exibirá a mensagem: `Started GestaoVagasApplication in X.XXX seconds`.

2. Acessando o Swagger UI

Abra o navegador e acesse: ➡ `http://localhost:8080/swagger-ui.html`

Você verá todos os 5 endpoints prontos para serem testados interativamente!

Exemplo de Testes via cURL / Postman:

A. Cadastrar uma Vaga (`POST /api/vagas`)

```
curl -X POST http://localhost:8080/api/vagas \
-H "Content-Type: application/json" \
-d '{
  "titulo": "Desenvolvedor Java Sênior",
  "descricao": "Atuação em microsserviços com Spring Boot e Cloud",
  "salario": 14000.00
}'
```

Resposta (HTTP 201 Created):

```
{
  "id": "7f8b9c0d-1234-5678-9abc-def012345678",
  "titulo": "Desenvolvedor Java Sênior",
  "descricao": "Atuação em microsserviços com Spring Boot e Cloud",
  "salario": 14000.0,
  "dataCriacao": "2026-08-26T16:00:00"
}
```

B. Listar Vagas Paginadas (GET /api/vagas)

```
curl -X GET "http://localhost:8080/api/vagas?page=0&size=5&sort=salario,desc"
```

C. Testando Validação de Erro (POST /api/vagas com corpo inválido)

```
curl -X POST http://localhost:8080/api/vagas \
-H "Content-Type: application/json" \
-d '{
  "titulo": "",
  "salario": -500.0
}'
```

Resposta (HTTP 400 Bad Request):

```
{
  "timestamp": "2026-08-26T16:05:00",
  "status": 400,
  "error": "Erro de validação",
  "message": {
    "titulo": "O título é obrigatório.",
    "salario": "O salário deve ser um valor positivo."
  },
  "path": "/api/vagas"
}
```

Conclusão e Próximos Passos

Parabéns! 🎉 Você acabou de aprender como construir uma API RESTful corporativa e robusta com Spring Boot 3!

O que você aprendeu neste tutorial:

1. Configurar um projeto Spring Boot do zero com Maven.
2. Organizar uma aplicação em camadas (**Controller**, **Service**, **Repository**, **Entity**, **DTO**, **Mapper**).
3. Utilizar o **Spring Data JPA** para operações em Banco de Dados sem escrever SQL puro.
4. Validar dados de entrada com **Jakarta Validation** e **Java Records**.
5. Mapear objetos com **MapStruct** e automatizar getters/setters com **Lombok**.
6. Tratar exceções de forma global com `@RestControllerAdvice`.
7. Documentar e testar endpoints com o **Swagger UI**.
8. Escrever e rodar testes unitários com JUnit 5 e Mockito.

Sugestões para continuar evoluindo:

- Implementar autenticação e autorização com **Spring Security** e **JWT**.
- Containerizar a aplicação criando um `Dockerfile` e `docker-compose.yml`.
- Adicionar filtros e busca avançada por título ou faixa salarial nas vagas.